

CMPE679: Deep Learning

Lab 3

Audio Generation Using WaveGAN for Spoken Digit Synthesis

Gloria Mbaka
Performed: 4/10/2026
Submitted: 4/12/2026

Instructor: Dr. Liu, Dongfang
TA: Ishan Renjen

By submitting this report, I attest that I neither have given nor have received any unauthorized assistance (including writing, collecting data, plotting figures, tables, or graphs, or using previous student reports as a reference), and I further acknowledge that giving or receiving such assistance will result in a failing grade for this course.

Your Signature: CGM

Abstract

This laboratory implemented a generative adversarial framework for raw audio synthesis using WaveGAN. The goal of the work was to learn a mapping from a low-dimensional random latent vector to a one-second waveform belonging to the distribution of spoken digits in the SC09 dataset. Unlike a conventional supervised classifier, this task did not ask the model to predict a label from an input sample. Instead, it required the generator to produce previously unseen waveform signals whose global temporal structure, local oscillations, and perceptual character resemble real human speech. Because adversarial training is highly unstable, the implementation was built around the Wasserstein GAN with gradient penalty formulation, together with checkpointing, background execution, resume-safe training, and periodic export of generated samples for visual and auditory inspection.

Training began with outputs that were indistinguishable from noise. As optimization progressed, the saved waveforms and spectrograms gradually developed structured temporal envelopes and speech-like energy patterns. Late-epoch listening showed that the generator could produce partially intelligible spoken digits, although residual background noise remained. To avoid relying only on subjective listening, a separate digit classifier was trained on the same SC09 dataset and used to evaluate generated samples. That classifier achieved a test accuracy of 97.2 percent on real data, which made it a credible proxy for assessing whether generated clips contained recognizable digit structure. By the final stage of training, most monitored generated samples were classified as valid digits with strong confidence, confirming that the WaveGAN model had learned meaningful audio structure even though perceptual quality did not become fully clean.

1. Introduction

Machine learning in its standard supervised form is often described as the process of finding a function with unknown parameters, defining a loss on training data, and optimizing the parameters to minimize that loss. In classification, the desired output is a class label. In regression, the desired output is a scalar. Generative modeling changes this objective in a fundamental way. Instead of learning a direct mapping from input to label, the model must learn to generate samples drawn from a target data distribution. In the GAN setting, the generator receives a random vector z sampled from a simple known distribution, usually a Gaussian distribution, and transforms it into a sample that should look, sound, or otherwise behave as though it came from the real dataset. This distinction makes audio generation especially difficult. A one-second waveform at 16 kHz contains sixteen thousand time-domain values. Those values are not independent. They jointly encode amplitude trajectories, spectral content, onset and offset behavior, phonetic transitions, and speaker-specific variations. Small local mistakes can create large perceptual artifacts. In image generation, artifacts may appear as small texture errors or checkerboard patterns. In audio

generation, artifacts often appear as harsh noise, metallic ringing, buzzing, or unintelligible speech. The problem is therefore not only high dimensional but also tightly linked to human perception. WaveGAN is one of the earliest important architectures for direct raw waveform generation. Conceptually, it is closely related to DCGAN, but it replaces two-dimensional convolutional operators with one-dimensional operators so that the model works directly on audio signals rather than images.

The lab handout emphasized that the main technical challenges in this assignment are proper preprocessing of the audio, implementation of the Wasserstein critic with gradient penalty, and addition of phase shuffle in the discriminator to reduce the critic's ability to exploit convolutional upsampling artifacts. It also emphasized that generated audio quality, rather than a single scalar metric, is the main basis of evaluation for this lab. Those instructions shaped the entire workflow of the project. The purpose of this report is therefore not only to state that a WaveGAN model was trained, but also to explain how the model was implemented, what modifications and debugging steps were required to make training practical in a limited cloud environment, how training behavior changed across epochs, and how qualitative listening was combined with quantitative classifier-based evaluation to determine whether the generated samples had moved beyond noise and into recognizable speech.

3. Dataset Description and Preprocessing

The dataset used in this laboratory was the SC09 spoken-digit subset derived from Google Speech Commands. It contains short recordings of the spoken digits zero through nine. The audio clips are organized into folders named by class label, such as zero, one, two, and so on. This directory structure made it straightforward to use folder names as labels when training the auxiliary classifier. It also provided a simple and interpretable dataset for adversarial generation because each class corresponds to a familiar short utterance with limited duration and relatively constrained linguistic complexity.

The lab instructions recommended approximately one second of audio at a sampling rate of 16 kHz, corresponding to 16,384 samples per clip after slicing or padding. In the implementation, the data pipeline (Figure 1) standardized the audio to mono, resampled it when necessary, and padded or truncated each signal to the fixed target length. Standardization of input length is essential for a convolutional generator and critic because the model architecture assumes a fixed output size and a fixed discriminator input size. If the dataset is not normalized into a common representation, the model is asked to learn a distribution that includes differences caused by preprocessing inconsistency rather than true acoustic variability.

For the WaveGAN model itself, the input to the critic remained raw waveform audio. However, for classifier training, a different representation was chosen. The classifier was trained on mel-spectrograms derived from the real SC09 waveforms. This was done because mel-spectrograms compress the waveform into a representation that more directly exposes the distribution of energy across time and frequency, which is highly useful for speech discrimination. As a result, the generator and classifier operated on different views of the same underlying signals: the generator learned to synthesize raw waveforms, while the classifier learned to recognize digit classes from frequency-domain structure. The preprocessing pipeline diagram is shown in figure 1.

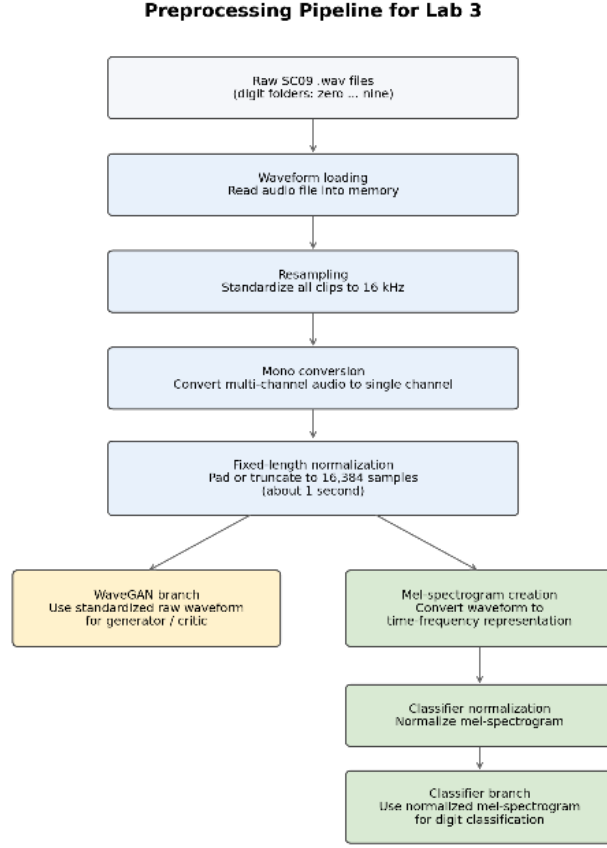


Figure 1. Preprocessing pipeline diagram showing waveform loading, resampling, mono conversion, fixed-length slicing, and mel-spectrogram creation for the classifier.

4. Theoretical Background

4.1 Generative Adversarial Learning

A GAN contains two neural networks trained in opposition. The generator G receives a latent vector z drawn from a simple distribution and produces a fake sample $G(z)$. The discriminator, or critic in the Wasserstein setting, receives either a real data sample x or a generated sample $G(z)$ and outputs a scalar score. The discriminator attempts to assign larger values to real samples and smaller values to generated samples. The generator attempts to produce samples that force the discriminator to fail. This adversarial game drives the generator toward the real data distribution.

$$z \sim N(0, I), \quad x_{fake} = G(z)$$

In the original GAN formulation, the min-max objective is expressed as

$$\min_G \max_D V(G, D) = E_{\{x \sim P_{data}\}}[\log D(x)] + E_{\{z \sim P_z\}}[\log(1 - D(G(z)))]$$

This objective is linked to the Jensen-Shannon divergence between the data distribution and the generator distribution. However, when those distributions have little overlap, especially early in training, the discriminator can become too strong and gradients passed to the generator become uninformative. This is one reason standard GAN training is often unstable.

4.2 Wasserstein GAN and Gradient Penalty

To address this problem, the laboratory used the Wasserstein GAN with gradient penalty. In WGAN, the discriminator becomes a critic that estimates a quantity related to the Wasserstein distance, also called Earth Mover distance, between real and generated distributions. Intuitively, this distance reflects how much mass would need to be moved, and how far, in order to transform one distribution into the other. Unlike the Jensen-Shannon divergence, the Wasserstein distance provides a smoother and more informative training signal when the two distributions are far apart. The critic loss can be written in simplified form as the difference between fake and real critic scores plus a gradient penalty term:

$$L_D = E[D(G(z))] - E[D(x)] + \lambda E \left[\left(\left\| \nabla_{\{x_{hat}\} D(x_{hat})} \right\|_2 - 1 \right)^2 \right]$$

The generator loss is

$$L_G = -E[D(G(z))]$$

The gradient penalty forces the critic to satisfy the Lipschitz constraint required by the Wasserstein formulation. Without that regularization, the critic can develop gradients that are either too small or too large, making training unstable. The lab handout explicitly highlighted this point and explained that WGAN-GP was introduced to reduce exploding and vanishing gradient behavior in GAN training.

4.3 Why Audio GANs Are Hard

The course GAN lectures emphasized that generative modeling is fundamentally more difficult than standard discriminative classification. This difficulty increases for sequence generation because raw waveform output is continuous, high dimensional, and sensitive to perceptual artifacts. Even if the model captures broad structure, details such as consonant sharpness, voicing, and background smoothness may remain poor. The same lectures also stressed that GAN loss alone does not directly measure perceptual quality. Consequently, this project relied on saved samples, waveform plots, spectrogram analysis, and later a classifier to determine whether the generator was learning meaningful digit structure rather than merely optimizing an abstract scalar objective.

5. Model Architecture

5.1 Generator

The generator followed the WaveGAN design pattern. It began with a latent vector of dimension 100. A fully connected layer first projected that vector into a high-channel low-resolution representation. That representation was then reshaped into a one-dimensional feature tensor and progressively upsampled through a sequence of transposed one-dimensional convolution layers. Each stage increased temporal resolution while decreasing channel depth. Batch normalization and ReLU activation were used in the intermediate generator layers, and the final output layer used tanh so that the synthesized waveform values were constrained to the interval $[-1, 1]$.

This architecture mirrors the principle used in image-based deconvolutional GANs. The model starts with a compact abstract encoding of the sample and gradually expands it into a full-resolution

signal. For audio, this means the network must learn not only broad envelope structure but also fine oscillatory detail in the final stages. This is one reason late-stage generator layers are especially important for perceived sound quality.

5.2 Critic

The critic used a cascade of one-dimensional convolution layers that progressively downsampled the input waveform. Instead of outputting a probability, the final layer produced a scalar score. In the Wasserstein framework, this scalar is interpreted as part of the critic's estimate of distance between the real and generated distributions. The critic's job is therefore different from that of a standard binary discriminator. It is not only labeling samples as real or fake. It is learning a scoring function whose difference in expectation across real and fake samples provides a useful optimization signal.

5.3 Phase Shuffle

One distinctive component of WaveGAN is phase shuffle. During upsampling with transposed convolutions, the generator can introduce regular periodic artifacts. A discriminator without any temporal invariance can learn to detect these artifacts too easily. Phase shuffle applies a small random temporal shift to discriminator activations, making it harder for the critic to exploit exact alignment of those artifacts. In practice, this forces the critic to focus more on broader acoustic realism and less on trivially repeated structure.

Early in experimentation, the model was trained without phase shuffle, but later runs enabled it because the lab instructions strongly suggested that phase shuffle is a meaningful refinement for WaveGAN. Once enabled, the generated spectrograms showed better structural development and the listening results became more speech-like, even though the final outputs were still noisy.

6. Training Procedure and Implementation Workflow

Training a GAN for audio generation is not a short single-pass experiment. The implementation had to be built to survive long runtimes, interruptions, and debugging cycles. The notebook and training script were therefore organized with checkpointing, logging, generated sample export, and resume support. This allowed the run to continue after interruptions and made it possible to compare outputs across many epochs without retraining from scratch.

The practical workflow proceeded in stages. First, a basic working model was trained to ensure that the generator and critic produced outputs of the expected shapes and that gradient penalty was correctly computed. Second, phase shuffle was enabled and critic steps were tuned. Third, training was launched in the background using `nohup` so that it could continue running inside Lightning AI even when the interactive notebook session was not actively watched. Fourth, generated waveform and spectrogram plots were saved at regular checkpoints, making it possible to inspect the same sample index across epochs. This matters because using fixed noise during monitoring allows the user to see whether a specific latent vector produces progressively more realistic output over training.

Several optimization decisions were adjusted over the course of the project. Early training used more aggressive settings, but later fine-tuning runs reduced the learning rates and the number of critic steps. This was done because once the model had already learned broad speech structure, the

goal shifted from learning the distribution coarsely to refining outputs. Lower learning rates made the updates smaller and less destructive. Reducing the number of critic updates decreased the risk that the critic would overpower the generator late in training.

The final extended training run reached epoch 115. Late training did not dramatically change perceptual quality compared with epochs around the low nineties, but it preserved and slightly refined the recognizable digit structure.

6.1 Hyperparameters and Runtime Settings

The core hyperparameters used in the major training runs included a batch size of 128, latent dimension 100, sample rate 16 kHz, and slice length 16,384. Critic steps were initially higher and later reduced during fine-tuning. Learning rates were lowered in the later run to $1e-5$ for both generator and critic when the training objective became refinement rather than coarse learning. Generated preview samples were saved regularly so that the evolution of a fixed set of latent vectors could be inspected.

7. Auxiliary Digit Classifier for Evaluation

One of the major difficulties in Lab 3 is evaluation. The handout stated directly that generated audio quality is not easy to quantify and that listening quality is the main metric. That is true, but qualitative listening alone is subjective and can be inconsistent across runs. To strengthen the evaluation, a separate supervised digit classifier was trained on the real SC09 dataset and later used to classify generated samples. The first attempt used a simple raw-waveform classifier, but its convergence was weak and the resulting predictions were not sufficiently reliable. The classifier was then redesigned to use mel-spectrogram inputs and a deeper two-dimensional convolutional architecture. This choice improved convergence substantially because mel-spectrograms expose speech frequency structure more directly than raw waveform inputs. The final classifier included convolution, batch normalization, nonlinear activation, pooling, dropout, and a final linear classifier layer. The classifier was trained with an explicit train, validation, and test split. Best and last checkpoints were saved separately, and the best validation checkpoint was evaluated on the held-out test set. The final test accuracy reached 97.2 percent. This is a critical result because it validates the classifier itself before it is used as an evaluator for generated audio. A weak classifier cannot provide meaningful support for GAN evaluation. A classifier with 97.2 percent test accuracy on real data, while not perfect, is strong enough to give credible evidence that later generated samples contain real digit-like structure.

7.1 Why Classifier-Based Evaluation Was Needed

Several other automated approaches were explored but proved unsuitable. General-purpose speech recognition models such as Whisper were unreliable on these generated clips, partly because the clips were short, noisy, and outside the kinds of natural speech conditions such models expect. Off-the-shelf labelers trained on broader command vocabularies also produced incorrect or irrelevant words because their label spaces did not match the digit-only SC09 setup. Training a custom classifier on the exact dataset of interest solved this mismatch problem and gave an evaluation method aligned with the generator's training domain.

8. Results and Analysis

8.1 Evolution of Generated Waveforms and Spectrograms

WaveGAN generator and critic loss curves across training is shown in Figure 2. The earliest generated waveforms had the expected behavior of an untrained or barely trained GAN: they looked like random noise with no coherent envelope and no obvious temporal organization as shown in Figure 3. Spectrograms (Figure 4) at that stage were diffuse and lacked the concentrated energy bands associated with speech. As training continued, the waveforms began to develop clearer regions of high amplitude separated by quieter regions, indicating that the generator was learning gross onset and offset structure rather than producing uniform broadband noise.

Mid-epoch spectrograms showed an especially important qualitative change. Instead of only diffuse energy, the generated spectrograms developed vertical and banded regions more consistent with voiced segments and transient speech events. Although the outputs remained noisy, the plots no longer looked like arbitrary noise fields. They began to show the kind of structured energy concentration expected from short spoken digits. This visual trend aligned with listening observations that the clips sounded increasingly human, even when individual words were not yet fully clear.

By late epochs, especially from around epochs 90 to 115, several clips were clearly perceived as human voice by listening. The model had therefore crossed the most important threshold for this lab: it was no longer generating only noise. Instead, it was generating speech-like signals with recognizable digit content. The remaining limitation was not the absence of structure, but the persistence of noise and imperfect articulation. WaveGAN generator and critic loss curves across training is shown in Figure 2.

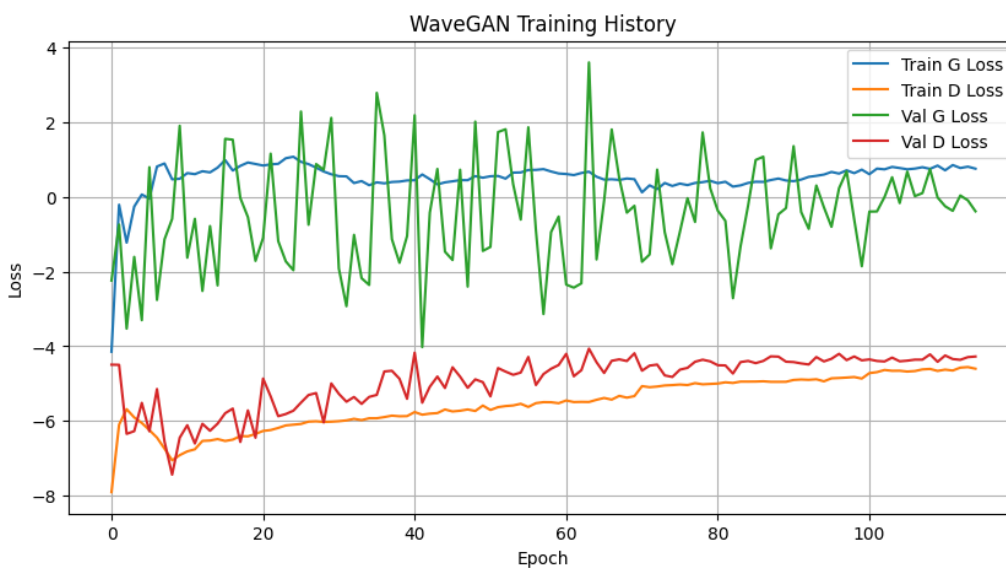


Figure 2. WaveGAN generator and critic loss curves across training.

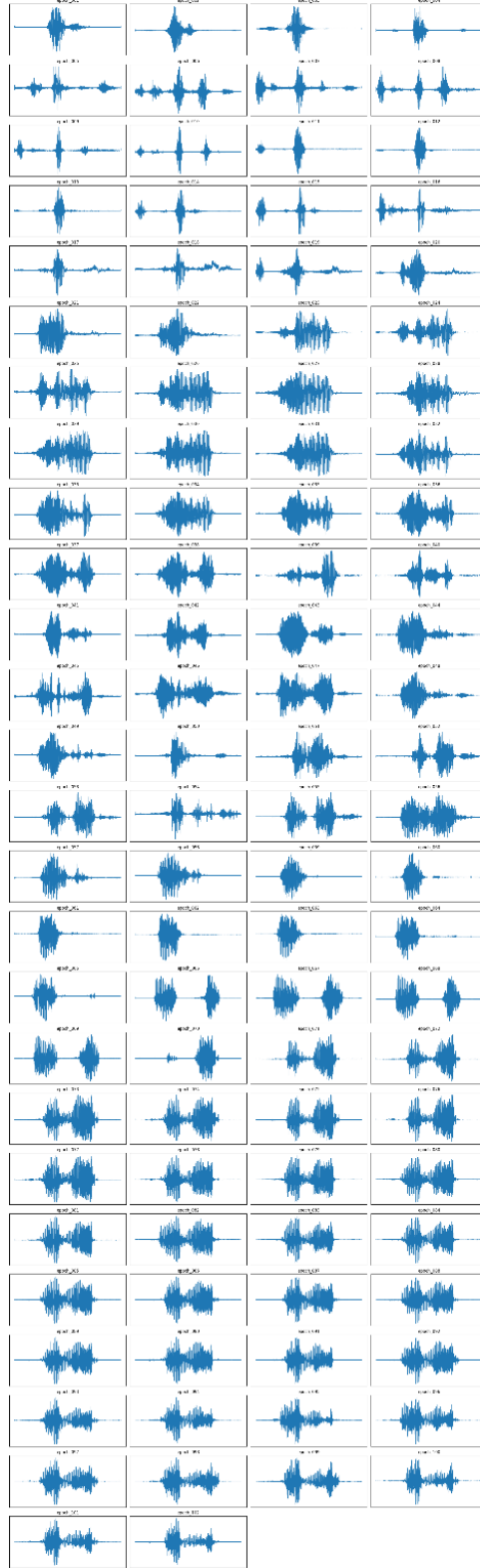


Figure 3. Waveform progression across epochs for sample_4 showing how the same sample index evolves from early noisy output to later speech-like structure.

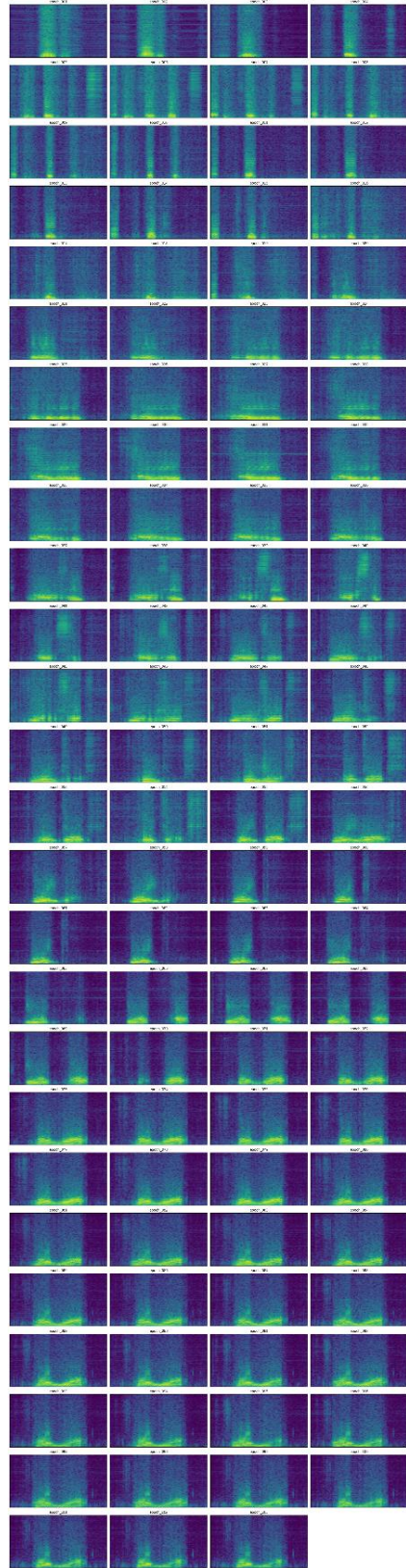


Figure 4. Spectrogram progression across epochs for sample_4 showing how energy distribution becomes more speech-like over training.

8.2 Final Classifier-Based Evaluation of Generated Samples

The strongest late-stage evidence came from the classifier predictions on generated samples (Table1). At epoch 115, the monitored sample set was classified as follows: sample 1 was identified as six with confidence 0.999, sample 2 as three with confidence 0.766, sample 3 as four with confidence 0.848, sample 4 as one with confidence 0.819, sample 5 as one with confidence 0.985, sample 6 as five with confidence 0.535, sample 7 as five with confidence 0.995, and sample 8 as six with confidence 0.949. These results show that most samples were not only assigned to valid digit classes but were assigned with high confidence.

Table 1: Final classifier-based evaluation of generated samples summarizing generated sample predictions and confidences for the final epoch 115.

Sample	Predicted Digit	Confidence
1	six	0.999
2	three	0.766
3	four	0.848
4	one	0.819
5	one	0.985
6	five	0.535
7	five	0.995
8	six	0.949

It is important to interpret these numbers correctly. A high-confidence classifier prediction does not by itself guarantee that the sample is perfect audio. However, when a classifier that achieves 97.2 percent test accuracy on real SC09 data repeatedly assigns plausible digit labels with strong confidence to generated clips, it provides convincing evidence that the generator has learned class-consistent acoustic structure. This result is much stronger than a vague statement that the output sounds somewhat speech-like.

At the same time, the confidence spread also reveals the remaining weakness. Sample 6 at 0.535 and earlier weak predictions in other epochs indicate that some clips remain ambiguous. This is consistent with listening, where some outputs sounded like distorted versions of digits or contained noticeable noise. Therefore, the correct conclusion is not that the model achieved clean speech synthesis, but that it achieved partially intelligible digit generation with strong evidence of learned structure.

8.3 Relationship Between Loss and Perceived Quality

One of the most important insights from this lab is that GAN losses cannot be interpreted in the same way as supervised classification loss. In a classifier, a decreasing loss often corresponds fairly directly to improved predictive behavior. In a GAN, the generator and critic are solving a min-max game. A change in one loss can reflect changes in the opponent rather than clean monotonic progress in sample quality. For this reason, late-epoch model selection was based on a combination of listening, spectrogram structure, and classifier output rather than by choosing the numerically smallest generator loss alone.

The test generator loss at the final stage was -0.3316 and the test critic loss was -4.5107. These values indicate a stable late-stage adversarial equilibrium rather than catastrophic divergence, but they do not by themselves prove that one late epoch is perceptually better than another. In practice, the latest checkpoint was adopted as the final base model because the later outputs sounded better and because the classifier results remained strong through the final epoch.

9. Issues Encountered and How They Were Resolved

Several important implementation issues were encountered during this lab, and documenting them is necessary because they shaped the final result. The first issue was the difficulty of adversarial training itself. Early outputs remained noisy for many epochs, and it was not immediately obvious whether the generator was learning meaningful structure or merely producing structured noise. This was addressed by saving the same sample indices across epochs, examining spectrogram progression, and listening to the generated clips directly.

A second issue involved evaluation. Generic speech recognition tools did not provide trustworthy results because the generated clips were noisy and the domain was highly restricted to short spoken digits. Instead of forcing an unsuitable tool to work, the workflow was changed to a domain-matched classifier-based evaluation strategy. This was a significant improvement in experimental design because it aligned the evaluator with the actual data distribution.

A third issue was that an early simple classifier design was not converging sufficiently well. The solution was to redesign the classifier around mel-spectrogram inputs and a stronger convolutional backbone. This led to high validation and test performance and turned the classifier into a useful evaluator rather than a weak secondary model. Finally, even after the generator had learned clear digit structure, output noise stopped improving meaningfully. At that point, further training on the same architecture did not appear to offer major gains, which led to the conclusion that the model had effectively reached the useful limit of this WaveGAN setup.

10. Discussion and Potential Improvements

The central technical lesson from this lab is that learning a generative model of raw waveforms is substantially harder than training a discriminative model on the same dataset. The classifier reached 97.2 percent test accuracy, while the generator, despite long training, still produced noisy speech. This asymmetry is not surprising. In classification, the model only needs to identify decision boundaries between classes. In generation, the model must synthesize realistic samples from the entire data distribution, including details that human listeners can detect immediately when they are wrong.

A natural question is what could be done to improve sound quality beyond the final model obtained here. One possibility is to increase model capacity, for example by enlarging the generator and critic. However, that would require retraining from scratch because checkpoint weights from one architecture cannot be safely reused in a different architecture with incompatible layer dimensions. Another possibility is to move away from raw waveform generation entirely and use a spectrogram-based generator combined with a neural vocoder. That approach is more common in modern speech synthesis because it places the learning problem into a representation that is easier to model cleanly.

Feature matching losses, multi-scale discriminators, and stronger regularization are also promising directions. In addition, post-processing methods such as light denoising or band-pass filtering can improve perceived quality for presentation, although they do not change the underlying generator. For the purposes of this laboratory, correct conclusion is that the model succeeded in learning digit-level structure from raw audio but did not fully solve the harder problem of clean speech synthesis.

11. Conclusion

This laboratory implemented and trained a WaveGAN model for raw audio generation on the SC09 spoken-digit dataset. The final model did not achieve clean speech synthesis, but it did achieve the more important threshold required by the lab: it generated clear human-voice-like signals containing recognizable digit structure.

The final evidence for success comes from the combination of three observations. First, the late-epoch waveforms and spectrograms developed consistent speech-like structure rather than remaining diffuse noise. Second, listening confirmed that several outputs sounded like human speech and in some cases like specific digits. Third, the auxiliary classifier, validated at 97.2 percent test accuracy on real data, assigned plausible digit labels to most generated samples with high confidence. Taken together, these results show that the WaveGAN model learned a meaningful approximation of the spoken-digit data distribution.

References

- [1] C. Donahue, J. McAuley, and M. Puckette, “Adversarial Audio Synthesis,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2019.
- [2] I. Goodfellow *et al.*, “Generative Adversarial Nets,” in *Advances in Neural Information Processing Systems*, 2014, pp. 2672–2680.
- [3] M. Arjovsky, S. Chintala, and L. Bottou, “Wasserstein GAN,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2017, pp. 214–223.
- [4] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, “Improved Training of Wasserstein GANs,” in *Advances in Neural Information Processing Systems*, 2017, pp. 5767–5777.
- [5] D. Liu, “GAN,” course lecture slides, CMPE 679 Deep Learning, Rochester Institute of Technology, Rochester, NY, USA.
- [6] I. Renjen, “CMPE-679 Lab 3: WaveGAN,” lab handout, Rochester Institute of Technology, Rochester, NY, USA, Mar. 17, 2026.
- [7] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems*, 2019, pp. 8024–8035.
- [8] P. Warden, “Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition,” *arXiv preprint arXiv:1804.03209*, 2018.
- [9] A. Radford, L. Metz, and S. Chintala, “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks,” *arXiv preprint arXiv:1511.06434*, 2015.
- [10] A. Odena, V. Dumoulin, and C. Olah, “Deconvolution and Checkerboard Artifacts,” *Distill*, 2016.